

# First Normal Form (1NF)

Complete Study Guide & Exam Notes • Atomicity, Multivalued Attributes & Decomposition Methods

## 1. What is First Normal Form (1NF)?

**First Normal Form (1NF)** is the foundational level of database normalization in Relational Database Management Systems (RDBMS). According to the relational model formulated by E.F. Codd, a relation must contain atomic (indivisible) values in every cell of a table.

### ★ EXAM DEFINITION

A relation is in **First Normal Form (1NF)** if every attribute contains only **atomic (single, indivisible)** values from its domain, and the relation contains **no repeating groups or multivalued attributes**.

**In Simple Terms:** One Cell → One Value

## 2. Understanding Multivalued Attributes

A **multivalued attribute** is an attribute that holds more than one value in a single cell for a given record. Storing multiple values in a single cell directly violates the core principle of atomicity.

### Non-1NF Example Table (Violates Atomicity)

| Roll_No | Name  | Course  |
|---------|-------|---------|
| 1       | Sai   | C, C++  |
| 2       | Harsh | Java    |
| 3       | Omkar | C, DBMS |

Here, the Course column contains comma-separated values (e.g., C, C++ and C, DBMS) in single cells. Hence, this table is **NOT in 1NF**.

### 🧠 KEY RULE TO REMEMBER

One cell containing multiple values → **Violates 1NF (Not Atomic)**.

## 3. Common Examples of Multivalued Attributes

Multivalued attributes frequently occur in unnormalized real-world data designs:

- **Phone Numbers:** Storing multiple contact numbers like "9876543210, 8765432109" in a single column.
- **Addresses:** Storing temporary and permanent locations together like "Delhi, Mumbai".
- **Courses / Skills:** Combining multiple enrollments or technical skills into one string like "C, C++, Java".

## 4. How to Convert a Non-1NF Table into 1NF?

There are three primary methods to achieve 1NF. However, they vary significantly in design quality and scalability.

### Method 1: Create Separate Rows (Row Expansion)

Duplicate row metadata (like ID and Name) and create an individual row for each distinct attribute value.

| Roll_No | Name  | Course |
|---------|-------|--------|
| 1       | Sai   | C      |
| 1       | Sai   | C++    |
| 2       | Harsh | Java   |
| 3       | Omkar | C      |
| 3       | Omkar | DBMS   |

#### Primary Key Analysis for Method 1:

- Roll\_No alone cannot be the primary key because values repeat (1, 1, 3, 3).
- Course alone cannot be the primary key because courses repeat.
- **Solution:** Use a **Composite Primary Key** consisting of (Roll\_No, Course).

### Method 2: Create Separate Attribute Columns

If there is a fixed upper limit of values (e.g., maximum 2 courses per student), add additional columns.

| Roll_No (PK) | Name  | Course_1 | Course_2 |
|--------------|-------|----------|----------|
| 1            | Sai   | C        | C++      |
| 2            | Harsh | Java     | NULL     |
| 3            | Omkar | C        | DBMS     |

#### ⚠ LIMITATION OF METHOD 2

Method 2 is **not scalable**. If one student enrolls in 20 courses, 20 columns are required (Course\_1 to Course\_20), leading to extensive NULL values and structural inefficiency.

### Method 3: Decompose into Two Related Tables ★ (Best Design)

Decompose the original unnormalized relation into two separate, normalized tables connected via a foreign key relationship.

Table 1: Student (Base Table)

| Roll_No (PK) | Name  |
|--------------|-------|
| 1            | Sai   |
| 2            | Harsh |
| 3            | Omkar |

Table 2: Student\_Course

| Roll_No (FK) | Course |
|--------------|--------|
| 1            | C      |
| 1            | C++    |
| 2            | Java   |
| 3            | C      |
| 3            | DBMS   |

## 5. Base Table & Relational Keys

In Method 3, Student(Roll\_No, Name) acts as the **Base (Parent) Table** holding master records. The Student\_Course table references it.

### RELATIONAL KEY MAPPING

**Student Table** [ Base / Parent ]

Roll\_No (Primary Key) • Name

↑

| (Foreign Key Reference)

↓

**Student\_Course Table** [ Child Table ]

Roll\_No (FK) • Course

**Composite Primary Key: (Roll\_No, Course)**

## SQL Schema Representation

```
CREATE TABLE Student (
  Roll_No INT PRIMARY KEY,
  Name VARCHAR(50)
);

CREATE TABLE Student_Course (
  Roll_No INT,
  Course VARCHAR(50),
  PRIMARY KEY (Roll_No, Course),
  FOREIGN KEY (Roll_No) REFERENCES Student(Roll_No)
);
```

## 6. Crucial Concepts & Comparisons

### Comparison of the Three Conversion Methods

| Method                | Representation                    | Primary Key           | Main Drawback / Feature                       |
|-----------------------|-----------------------------------|-----------------------|-----------------------------------------------|
| 1. Separate Rows      | Duplicate student rows per course | (Roll_No, Course)     | Data redundancy (Name repeated)               |
| 2. Separate Columns   | Course_1, Course_2, ...           | Roll_No               | Not scalable; excess NULL values              |
| 3. Decomposition<br>★ | Two linked tables (Parent-Child)  | Composite PK in child | <b>Best practice:</b> Scalable & clean design |

#### ⚠ IMPORTANT DISTINCTION: 1NF VS PRIMARY KEY

A table having a Primary Key does **NOT** automatically imply it is in 1NF. For example, Student (SID [PK], Name, Course) with Course = "C, C++" has a valid Primary Key (SID), yet it violates 1NF due to non-atomic values.

## 7. Exam-Ready Answer (10 Marks Format)

### Q. Explain First Normal Form (1NF) with an example. How do you convert a table into 1NF?

#### 1. Definition:

First Normal Form (1NF) is the foundational step in database normalization. A relation is in 1NF if and only if all attributes contain *atomic (single, indivisible) values* and there are *no repeating groups or multivalued attributes*.

#### 2. Unnormalized Table Example (Not in 1NF):

| Roll_No | Name  | Course |
|---------|-------|--------|
| 1       | Sai   | C, C++ |
| 2       | Harsh | Java   |

*Reason:* The Course attribute contains multiple values (C, C++) in a single cell for Roll\_No = 1.

#### 3. Conversion into First Normal Form (Method 1 - Atomic Rows):

| Roll_No | Name  | Course |
|---------|-------|--------|
| 1       | Sai   | C      |
| 1       | Sai   | C++    |
| 2       | Harsh | Java   |

*Primary Key:* (Roll\_No, Course) forms a composite primary key.

#### 4. Recommended Decomposition (Method 3 - Relational Solution):

To avoid data redundancy, the relation is decomposed into two tables:

- **Student**(Roll\_No, Name) — *Base Table with Primary Key Roll\_No*
- **Student\_Course**(Roll\_No, Course) — *Child Table with Foreign Key Roll\_No referencing Student(Roll\_No)*

#### 5. Key Summary Points:

- 1NF requires strict atomicity.
- Multivalued attributes must be eliminated.
- Having a primary key does not automatically guarantee 1NF.
- Relational decomposition provides the most scalable design.

### ONE-LINE REVISION SUMMARY

**1NF = Every cell in every row must contain a single, atomic value.**